

Loom & Link:

A Real-time Synchronized Scheduler for Long Distance Relationships

Technical Documentation & Architecture Design

Syncing Hearts Across Timezones

Prepared for: Babeş-Bolyai University Academic Review
Author: Tompea Maria Daria
Date: February 2027

Contents

1	Introduction	2
1.1	Project Vision	2
1.2	Core Objectives	2
2	Design Philosophy	3
2.1	The “Cute” Aesthetic	3
2.1.1	Color Palette	3
2.2	Interface Layout	3
3	Technical Stack	4
3.1	Frontend: Python with Flet	4
3.2	Backend: Supabase Real-time	4
4	Timezone Synchronization	5
4.1	The UTC Standard	5
4.2	Conversion Algorithm	5
4.3	Day Boundary Logic	5
5	Application Modules	6
5.1	Authentication & Pairing	6
5.2	Event Management	6
6	Database Schema	7
6.1	Entity Definitions	7
6.1.1	User Table	7
6.1.2	Event Table	7
7	Implementation Code	8
7.1	The Table Component	8
7.2	Weekly Reset Logic	8
8	Conclusion	9

Introduction

1.1 Project Vision

Maintaining a relationship across vast geographic distances introduces a unique set of challenges, primarily revolving around temporal coordination. “Loom & Link” is a Python-based application designed to bridge this gap by providing a visual, interactive, and “cute” scheduling environment. Unlike generic calendar apps, Loom & Link focuses on the emotional and visual connection between two partners.

1.2 Core Objectives

The primary goals of the system are:

- **Real-time Sync:** Every action taken by one partner is instantly visible to the other.
- **Transparent Localization:** Automated timezone conversion that renders the same event correctly on two different local grids.
- **High Aesthetic Appeal:** Using modern UI principles to create a warm, inviting environment.
- **Simplicity:** A code-based pairing system that eliminates complex email-based invites.

Design Philosophy

2.1 The “Cute” Aesthetic

The application utilizes a **Glassmorphism** design language. This involves blurred background elements, semi-transparent panels, and soft shadows to create a sense of depth and warmth.

2.1.1 Color Palette

We have curated a 10-color pastel palette for event classification, ensuring that the schedule remains vibrant yet readable:

Color Name	HEX Code	Typical Activity
Strawberry	#FFB7B2	Date Night
Sky Blue	#B2E2F2	Online Gaming
Mint Green	#B2F2BB	Studying Together
Lavender	#D1B2F2	Sleep/Wind Down
Lemon	#FFF1B2	Morning Call

Table 2.1: Subset of the 10-color palette logic.

2.2 Interface Layout

The main interface consists of a weekly grid. The Y-axis represents time in 30-minute increments, while the X-axis represents the days of the week.

Technical Stack

3.1 Frontend: Python with Flet

The application is built using **Flet**, a framework that enables the creation of Flutter apps using Python. This allows for a reactive UI that behaves like a native application on both desktop and mobile platforms.

3.2 Backend: Supabase Real-time

To facilitate the sharing of tables, **Supabase** is employed. It provides a PostgreSQL database with a real-time listener. When Partner A adds an event, the database emits a broadcast, which Partner B's client receives instantly.

Timezone Synchronization

4.1 The UTC Standard

To handle users in different regions (e.g., Italy and USA), the application stores all timestamps in the database as **UTC+0**.

4.2 Conversion Algorithm

When the application fetches an event E with a start time T_{utc} , it detects the user's local timezone TZ_{local} and performs the following calculation:

$$T_{local} = T_{utc} + \text{Offset}(TZ_{local}, T_{utc}) \quad (4.1)$$

4.3 Day Boundary Logic

A unique requirement of Loom & Link is handling events that span across different days for different users. For example:

- **Partner A (Rome, UTC+2):** Adds a “Movie Night” on Sunday at 11:00 PM.
- **Partner B (New York, UTC-4):** The same event occurs at 5:00 PM on Sunday.

The engine ensures the event appears on the Sunday column for both. However, if the time difference shifts the event to a Monday for one partner, the app intelligently renders it in the Monday column of the *viewer's* local week.

Application Modules

5.1 Authentication & Pairing

The login system uses a simple alphanumeric code ($6 \leq \text{length} \leq 15$).

```
1 def validate_code(code):  
2     if 6 <= len(code) <= 15:  
3         return True  
4     return False
```

Once logged in, a user can enter their partner's code to establish a bidirectional link in the `couples` table.

5.2 Event Management

The event popup allows users to input:

- Activity Name (e.g., “Dinner Date”).
- Description (e.g., “Eating sushi while on FaceTime”).
- Color Selection (One of 10 presets).
- Start and End Time (converted to UTC before saving).

Database Schema

6.1 Entity Definitions

6.1.1 User Table

Stores `user_id`, `secret_code`, and `region`.

6.1.2 Event Table

Stores `id`, `couple_id`, `title`, `desc`, `start_time`, `end_time`, and `color_index`.

Implementation Code

7.1 The Table Component

```
1 class ScheduleTable(ft.UserControl):  
2     def build(self):  
3         # Create 7 columns for the week  
4         # Populated with local-converted events  
5         pass
```

7.2 Weekly Reset Logic

At the start of each week (Monday 00:00 UTC), the application triggers a routine to archive the previous week's events and provide an empty table, as per user requirements.

Conclusion

Loom & Link provides a robust, aesthetically pleasing solution for long-distance coordination. By abstracting the complexities of timezone mathematics and offering a shared, colorful environment, it fosters closer connection through organized quality time.